# Semantically Transparent Language Generation for Cognitive Robots

**Marjorie McShane** MCSHAM2@RPI.EDU
**Jesse English** DRJESSEENGLISH@GMAIL.COM
**Sergei Nirenburg** NIRENS@RPI.EDU
**Christian Arndt** ARNDTC@RPI.EDU

Language-Endowed Intelligent Agents Lab, Rensselaer Polytechnic Institute, Troy, NY 12180, USA

Abstract

In order to be reliable collaborators, cognitive robots need to reason and communicate in semantically transparent ways. This paper details the implemented, human-inspired model of language generation used by the cognitive robots called LEIAs. The generation process is divided into two phases: (1) *content specification*, which includes triggering the generation process and formulating the content of the message in the ontologically grounded metalanguage of agent cognition; and (2) *language rendering*, which involves deciding how to express that meaning in a particular natural language. Whereas content specification is modeled symbolically and is wholly explainable, rendering is modeled neurosymbolically: its core processes are transparent and explainable, but certain low-risk support functions are outsourced to a language model. This paper describes language generation for LEIAs and illustrates system operation in both simulation and development environments.

## 1 Introduction

Cognitive robots that will serve as reliable assistants to humans must be adept at communicating using meaningful language. This means they must interpret language into, and generate language from, meaning representations written in the ontologically-grounded language of their cognition.[1] Preparing agents to communicate in this way requires computational cognitive modeling of the kind being pursued in the LEIA program of research. Whereas language *understanding* by LEIAs has been described extensively (e.g., McShane & Nirenburg, 2021; McShane, Nirenburg, & English, 2024), here we present the first comprehensive model of language *generation* — both content specification and language rendering.[2] *Content specification* is deciding what to say, and *language rendering* is deciding how to say it. The top level of the LEIA's language generation model is shown in Fig. 1.

[1] Motivation for having agents think in a metalanguage rather than using a natural language as a proxy is provided in Nirenburg and Raskin (2004) and McShane and Nirenburg (2021). In addition, the ever-growing body of work pointing out the limitations of language models makes it clear that we need a more reliable solution than outsourcing all aspects of AI to those models (Oruganti et al., 2026).

[2] Our past work on generation conceptually informed the work reported here; however, it did not model content specification and it was much more preliminary with respect to the many challenges of rendering (McShane & Leon, 2021; McShane, Nirenburg, & English, 2024; McShane et al., 2025b).

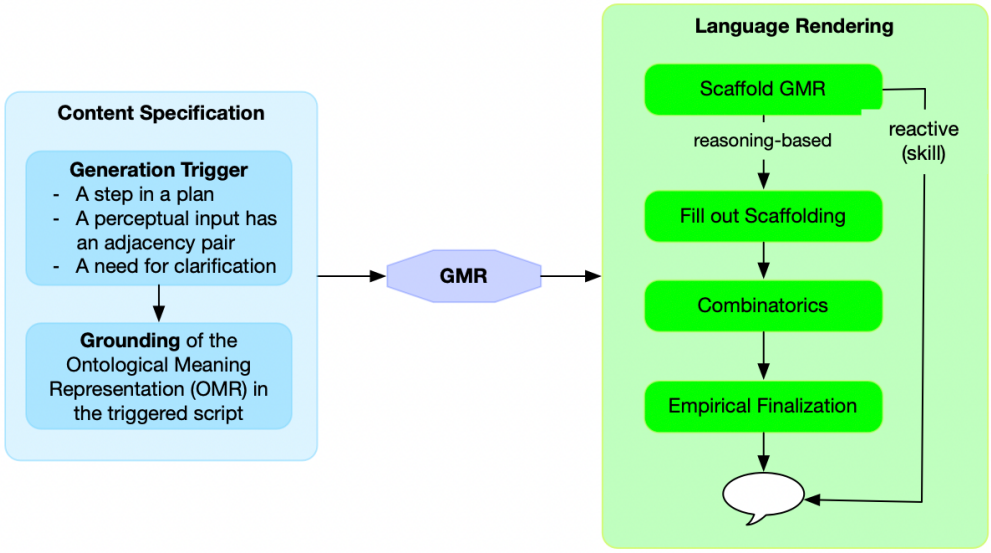



**Fig. 1** The top level of the model of language generation for LEIAs.

Since the audience at this conference is well versed in computational cognitive modeling and symbolic AI, we will proceed directly to the model of generation and present select background information as it becomes relevant.

## 2 Content Specification

Language generation begins with content specification, which is a reasoning process in which an agent decides *that* it wants to communicate something and *what* it wants to communicate. The output of content specification is generation meaning representation (GMR) that is written in the metalanguage of the LEIA's ontology.

Content specification is tightly guided by knowledge stored in, and procedures triggered from, the LEIA's ontology. Acquiring this knowledge and writing the associated programs is the work of developing knowledge-based systems: it is the price of reliability and explainability. Contrary to popular belief, such work is entirely feasible given today's technology, as long as it is guided by strong methodology of the type we will demonstrate shortly (see also McShane et al., 2026a,b).

As Fig. 1 shows, the fact that a LEIA wants to communicate something is triggered in one of three ways, which is handled by its agenda-oriented reasoning: (1) a step in plan it is pursuing involves a speech act; (2) a perceptual input requires a response, which is modeled using ontologically defined adjacency pairs such as REQUEST-INFO ~ RESPOND-TO-REQUEST-INFO; or (3) the agent does not understand an input sufficiently to act, which triggers it to seek clarification.[3] We will explain these eventualities using examples in a domain relevant for LEIA robots: **maintaining**

[3] See McShane et al. (2025a) for the model of actionability assessment that guides this assessment.

**shipboard equipment**. In working through the examples we will not skimp on presenting knowledge structures because symbolic modeling is all about knowledge. The examples are intended to convey that: (a) knowledge acquisition for LEIAs involves straightforward variations on a theme; (b) scripts (complex events) cover a wide range of eventualities thanks to the attached function calls that ground variables in the situation model; (c) the same ontological knowledge can be used by agent systems operating in any natural language; and (d) this approach to symbolic modeling can be replicated in other cognitive systems designed in the same spirit as LEIAs.

**Trigger 1: A step in a plan requires a speech act**. LEIAs understand their job, their world, and how to communicate with people thanks to scripts recorded in their ontology. The task described by the script MAINTAIN-EQUIPMENT has three subtasks: MONITOR-EQUIPMENT, DIAGNOSE-EQUIPMENT, and REPAIR-EQUIPMENT. In turn, MONITOR-EQUIPMENT has its own subtasks: READ-GAUGE-VALUE, LOG-GAUGE-VALUE, ASSESS-GAUGE-VALUE, and, if the reading is abnormal, then REPORT-GAUGE-ANOMALY. This last step requires communication.

Table 2 shows an abbreviated version of the script REPORT-GAUGE-ANOMALY. Comments following semi-colons show the values of the variables for the case when the agent is preparing to report that **the lube-oil pressure is a little low — 3.8 bar**.

**Table 2.** An excerpt from the script REPORT-GAUGE-ANOMALY. Comments following semi-colons show how underlined variables will be resolved to report that the lube-oil pressure is a little low — 3.8 bar.

| | | | | |
|---|---|---|---|---|
| REPORT-GAUGE-ANOMALY | | | | |
| IS-A | | SPEECH-ACT | | |
| AGENT | | LEIA-1 | | |
| BENEFICIARY | | TEAM-LEADER-1 | | |
| HAS-EVENT-AS-PART | | BUILD-GMR-1, RENDER-GMR-1 | | |
| CAUSED-BY | | PROPERTY-1.VALUE (RELATIVE-TO-NORM ≠ .5) | | |
| BUILD-GMR-1 | | | | |
| onto-procedure | | *ground-gmr* ASSERTIVE-ACT-1 | | |
| ASSERTIVE-ACT-1 | | | | |
| AGENT | | LEIA-1 | | |
| BENEFICIARY | | TEAM-LEADER-1 | | |
| THEME | *either* | 1. PROPERTY-1.RANGE | | ; PRESSURE-1 |
| | | 2. PROPERTY-1.RELATIVE-TO-NORM | | ; PRESSURE-1 |
| | | 3. ELABORATION-1 | | |
| | | DOMAIN | PROPERTY-1.RELATIVE-TO-NORM | ; PRESSURE-1 |
| | | RANGE | PROPERTY-1.RANGE | ; PRESSURE-1 |
| PROPERTY-1 | | | | |
| DOMAIN | | OBJECT-1 | | ; LUBE-OIL-1 |
| RANGE | | value-1 | | ; 3.8 (measured-in bar) |
| RELATIVE-TO-NORM | | value-2 | | ; .4 |

The indices appended to concept names distinguish *ontological instances*, which allows for cross-referencing across knowledge structures and the programs that use them. Like all speech acts, REPORT-GAUGE-ANOMALY has an AGENT (the speaker), a BENEFICIARY (the hearer), and two subtasks: BUILD-GMR for content specification, and RENDER-GMR for language rendering. Whereas

BUILD-GMR is specific to, and further detailed in, each individual script, RENDER-GMR plays out the same way across speech acts, so nothing more is said about it in individual speech acts.

BUILD-GMR-1 calls the procedure *ground-gmr,* which is stored in the ontology as a procedural attachment to the concept GROUND-GMR. *Ground-gmr* takes as its argument an ontological meaning representation (OMR[4]) and grounds (resolves) the variables within it. In the script REPORT-GAUGE-ANOMALY, the program *ground-gmr* takes as its argument the OMR headed by ASSERTIVE-ACT-1, which says that the LEIA will report to its TEAM-LEADER one of three things: either the actual property value that is of concern, how it relates to a normal value, or both. These options will be rendered respectively by utterances like `Lube oil pressure is at 3.8 bar / Lube oil pressure is a little low / Lube oil pressure is a little low — it's at 3.8 bar.`

The grounding process for speech acts can range from simple to complex. If converting an OMR into a GMR requires only looking up values of variables in predefined places, then the generic *ground-gmr* program suffices. However, if an OMR contains options, variables that need to be computed rather than looked up, and/or references that need to be expanded into new property-rich frames, then bespoke grounding functions are developed and appended to the given concept. This reflects an important methodological principle of LEIA modeling: knowledge engineers and system engineers collaborate closely. It was a misstep in early symbolic AI to build knowledge bases outside of the programs that would ultimately need to use them. Programs *also* capture knowledge — namely, how to operate over the static and dynamically compiled knowledge bases available to the agent.

Ontological knowledge is recorded as formally as feasible, to allow the programs to be as generic as possible. But we do not require knowledge engineers to immediately and definitively solve every knowledge representation problem, which would hamstring practical system building and is simply not necessary: Programs carry an equal share of the representational work in the LEIA ecosystem, and that balance should be exploited.

The knowledge in the concept RECORD-GAUGE-ANOMALY serves not only language generation but also language understanding. If someone says to an agent "Lube oil pressure is at 3.8 bar," the agent needs to figure out why they said it, which means identifying the intended speech act. The agent can do this by scanning through its SPEECH-ACT subtree and identifying a speech act whose value of CAUSED-BY aligns with its interpretation of this utterance.

When language generation is triggered by a step in a plan, as in the example we just worked through, the meaning must be expressed as a full proposition. By contrast, the other two triggers for language generation are tightly connected to a previous dialog move, facilitating the generation of realistic fragmentary utterances that a human would use in such situations. For example, if the chief engineer says, "Give me the wrench" but the LEIA sees two options, it's more natural to clarify by asking "The long one or the short one?", rather than "Should I give you the long one or the short one?" — or, much worse "Should I give you the long wrench or the short wrench?" The question is: Do people generate a full propositional meaning and then shorten it using a linguistic process, or do they generate a sub-propositional meaning and convey it directly in language? This

[4] All meaning representations in the LEIA ecosystem are interoperable, based on a generic XMR formalism. Additional specific fields are available for different types of meaning representations: text (TMRs), vision (VMRs), mental (MMRs), ontological (OMRs), and so on.

question is interesting both theoretically and practically. Following argumentation in McShane, Nirenburg, & Beale (2005), we model LEIAs to take the more streamlined approach. When LEIAs respond to utterances, they formulate only the meaning that fills the open slot in the dialog pair, and they express that directly in language. Their dialog model links the response to the utterance that triggered it, so they can always recreate the full propositional meaning that they are conveying in fragmentary form if called upon to do that.

Keeping this approach to modeling fragments in mind, we will now look at examples of the other two triggers for language generation, both of which involve responding to speech acts.

**Trigger 2: A perceptual input requires a response.** Consider the following example:

```
LEIA:  Lube oil pressure is at 3.8 bar.
Chief: What was the average during the last watch?
LEIA:  4.2 bar.
```

When the agent analyzes the Chief's question, it interprets it as a request to report a property value — that is, as an instance of the concept REQUEST-PROP-VALUE.[5] A basic principle of agent planning is that when the interpretation of a perceptual input has an adjacency pair stored in the ontology, the agent's next move is to instantiate the pair. So, detecting REQUEST-PROP-VALUE triggers an instance of its adjacency pair, RESPOND-TO-REQUEST-PROP-VALUE. The ontological description of the latter is shown in abbreviated form in Table 3. We indicate how grounding will play out in our example using comments following semi-colons.

**Table 3.** An excerpt from the script RESPOND-TO-REQUEST-PROP-VALUE. Comments following semi-colons show how underlined variables would be resolved to report a lube-oil pressure of 4.2 bar.

| | | | |
|---|---|---|---|
| **RESPOND-TO-REQUEST-PROP-VALUE** | | | |
| AGENT | LEIA-1 | | |
| BENEFICIARY | HUMAN-1 | | |
| HAS-EVENT-AS-PART | FIND-PROP-VALUE-1, BUILD-GMR-1, RENDER-GMR-1 | | |
| FIND-PROP-VALUE-1 | | | |
| onto-procedure | *find-calculate* PROPERTY-1 | | |
| BUILD-GMR-1 | | | |
| onto-procedure | *ground-gmr* ASSERTIVE-ACT-1 | | |
| ASSERTIVE-ACT-1 | | | |
| AGENT | LEIA-1 | | |
| BENEFICIARY | HUMAN-1 | | |
| THEME | PROPERTY-1 | | ; PRESSURE-1 |
| PROPERTY-1 | | | ; PRESSURE-1 |
| DOMAIN | OBJECT-1 | | ; LUBE-OIL-1 |
| RANGE | ANY-NUM-1 | | |
| ANY-NUM-1 | | | |
| VALUE | value-1 | | ; 4.2 |
| MEASURED-IN | value-2 | | ; bar |
| gen-guidance | *either* | 1. ANY-NUM-1.value | ; 4.2 |
| | | 2. ANY-NUM-1 | ; 4.2 (measured-in bar) |
| | | 3. pro-copula:ANY-NUM-1.value | ; pro-copula: 4.2 |
| | | 4. pro-copula: ANY-NUM-1 | ; pro-copula:4.2 (measured-in bar) |

[5] For the language understanding process that gives rise to this analysis, see McShane and Nirenburg (2021), McShane, Nirenburg, and English (2024), and McShane et al. (2025a).

In contrast to the previous script we saw, this one has an additional step before building and rendering the GMR: the agent has to identify or compute the requested property value. In this case, it has to compute it from stored values in its episodic memory. This process is triggered by the FIND-PROP-VALUE script. Once the agent knows the value it has to report, it builds the GMR in the same way as in our last example.

This example includes an extra field not shown earlier: gen-guidance. This provides metadata that will be passed to the language rendering system about what exactly to include in the utterance. Four options are offered: convey just the value itself (4.2); the value and the measuring unit (4.2 bar); a copular construction composed of a pronoun and the value (It was 4.2); or a copular construction composed of a pronoun, the value, and the measuring unit (It was 4.2 bar). Without this guidance, the language rendering system, which is generic, would unnecessarily generate many different candidate paraphrases of full propositions, including such things as "The average lube oil pressure during the last watch was 4.2 bar", which would be unnaturally verbose in response to "What was the average during the last watch?" Since we are relying on a language model to select the contextually best paraphrase, and since we cannot control its behavior, we constrain its choice space as narrowly as possible through symbolic modeling.

**Trigger 3: An input requires clarification.** Consider the following example:

```
Chief: Bring me the pipe wrench from the locker.
LEIA:  Which one?  or  The long one or the short one?
```

Agents always know the specific thing they need clarified because their language understanding script decorates candidate meaning representations with flags that point to the need for particular troubleshooting actions. Here, the agent needs referential disambiguation between two options, so it instantiates the script BINARY-REF-DISAMBIG.

The difference between this eventuality and the previous ones is that, in addition to providing language-independent gen-guidance that says "generate a conjoined noun phrase as the response", this script provides English-specific gen-guidance that offers two other options: one is variable-free ("Which one?") and the other is a combination of the variable-free and full-form options ("Which one: the short one or the long one?"). We will talk more about variable-free (reactive, skill-based) options in the next subsection. It might seem counterintuitive to record English-specific information in the language-independent ontology, but there is nothing wrong with that as long as there is a good reason for it (such recording a reactive response to a recognized concept) and the slots are named such that they can be used, ignored, or deleted in a particular language environment.

At this point, one might ask how many speech acts a LEIA will need in order to fulfill the requirements of a robotic application. The answer depends on what those requirements are as well as preferences of knowledge engineers. For example, there is no functional difference between embedding a list of cases within the ontological concept representing a broader speech act and splitting each case into its own speech act.

**Table 4.** An excerpt from the script BINARY-REF-DISAMBIG. Comments following semi-colons show how underlined variables would be resolved distinguish between a long and a short pipe wrench.

| | | |
|---|---|---|
| **BINARY-REF-DISAMBIG** | | |
| HAS-EVENT-AS-PART | FIND-CONTRASTIVE-VALUE-1, BUILD-GMR-1, RENDER-GMR-1 | |
| ID-DISTING-VALUE-1 | | |
| onto-procedure | *find-contrastive-value* (OBJECT-1, OBJECT-2) | |
| BUILD-GMR-1 | | |
| onto-procedure | *ground-gmr* REQUEST-INFO-1 | |
| REQUEST-INFO-1 | | |
| AGENT | LEIA-1 | |
| BENEFICIARY | HUMAN-1 | |
| THEME | DISJUNCTION-1 | |
| DISJUNCTION-1 | | |
| DOMAIN | OBJECT-1 | ; PIPE-WRENCH-1 |
| RANGE | OBJECT-2 | ; PIPE-WRENCH-2 |
| OBJECT-1 | | ; PIPE-WRENCH-1 |
| PROPERTY-1 | value-1 | ; LENGTH .8 |
| COREF | yes | |
| OBJECT-2 | | ; PIPE-WRENCH-2 |
| PROPERTY-1 | value-2 | ; LENGTH .2 |
| COREF | yes | |
| gen-guidance | NP-conj-fragment | |
| gen-guidance-Engl | | |
| If *find-contrastive-value* (OBJECT-1, OBJECT-2) fails, then output "Which one?" | | |
| Else if *find-contrastive-value* (OBJECT-1, OBJECT-2) succeeds, then output either | | |
| 1. GMR output from BUILD-GMR-1 | | |
| 2. "Which one?" | | |
| 3. "Which one: GMR output from BUILD-GMR-1?" | | |

## 3 Language Rendering

Language rendering creates natural language sentences from GMRs. Like all of agent cognition, it is modeled as an ontological script (LANGUAGE-RENDERING) whose top layer of subevents is shown below using numbers, and whose next layer is shown using bracketed lists.

1. GMR-SCAFFOLDING [REFLEXIVE-SCAFFOLDING, SHAPE-MATCH-SCAFFOLDING, DYNAMIC-SCAFFOLDING]
2. FILLING-OUT-SCAFFOLDING [GEN-TRANSFORM, FREEZE-HEAD-MOD-COMBOS, BUILD-MODIFIED-SCALARS, FINALIZE-GEN-SUBLEX, PREPARE-REFEX-GEN]
3. GMR-COMBINATORICS
4. FINALIZE-GENERATION [GEN-MORPHOLOGY, SENTENCE-SELECTION]

Although these concept names won't mean anything to readers yet, our point in presenting the skeleton of the script is to emphasize that the agent's knowledge about rendering is no different from its knowledge about anything else: it is recorded as a collection of ontological concepts, each

of which is defined by a combination of static knowledge and calls to the programs that operate over it.

In order to understand why language rendering is a complex process, one needs to understand some basic principles of how meanings map to language, which we will try to explain quickly using examples. Table 5 shows three simplified GMRs involving an orange bicycle, along with sentences that convey their meaning.[6] Notice that the bicycle's orange color is expressed in three different ways in English depending on what else is in the GMR. This is the first hint that converting GMRs into natural language is not simply a matter of looking up words in the lexicon and ordering them into a sentence.

**Table 5.** The overall content of a GMR matters for its language realization.

| | | |
|---|---|---|
| BICYCLE-2<br>COLOR orange | COST-1<br>DOMAIN BICYCLE-2<br>RANGE .8<br>BICYCLE-2<br>COLOR orange | AMUSE-1<br>CAUSED-BY COLOR-1<br>EXPERIENCER HUMAN-1<br>TIME past<br>COLOR-1<br>DOMAIN BICYCLE-3<br>RANGE orange |
| **The bicycle is orange**. | **The orange bicycle is expensive.** | **The fact that the bicycle was orange amused me.** |

Now consider the structure and content of senses in the LEIA's computational semantic lexicon. Below are three verbal senses that will be used in our next example: **Mary needed to feed Fido before going out to dinner.**

Table 6. Sample senses from the LEIA's lexicon.

| | | |
|---|---|---|
| need-v1<br>def *need* + xcomp<br>ex I needed to do my homework<br>syn-struc<br>subject $var1<br>verb $var0<br>xcomp $var2<br>sem-struc<br>MODALITY<br>TYPE obligative<br>VALUE 1<br>SCOPE ^$var2<br>^$var2<br>AGENT ^$var1<br>mpu xcomp-u (^$var2)<br>mpg xcomp-g (^$var2) | feed-v1<br>def To give food to; transitive<br>ex She fed the dog<br>syn-struc<br>subject $var1<br>verb $var0<br>directobject $var2<br>sem-struc<br>FEED<br>AGENT ^$var1<br>BENEFICIARY ^$var2 | go-v72<br>def phrasal: X go out to dinner<br>ex We're going out to dinner.<br>syn-struc<br>subject $var1<br>verb $var0<br>prep-part out<br>pp<br>prep to<br>obj dinner<br>sem-struc<br>EAT-AT-RESTAURANT<br>AGENT ^$var1 |

[6] We exclude details that would elucidate the choices of referring expressions: *the* (not *a*) *bicycle; me*.

Every lexical sense contains a syn-struc zone that records its syntactic constituents (including dependency structures) and a sem-struc zone that records the associated meanings as mappings to ontological concepts. Numerical indices coreference the variables in the syn-struc and sem-struc. "^" indicates "the meaning of the variable" — i.e., its concept mapping.

The lexicon contains only the basic diathesis of argument-taking words. For example, feed-v1 directly covers active voice sentences like "Mary fed Fido", and it needs to be dynamically adjusted to match the syntax of its passive counterpart, "Fido was fed by Mary". Such adjustments are called *transformations*. When a lexical sense predicts the need for a transformation, associated function calls are recorded in its *meaning procedures* zones — one for language understanding (mpu) and one for language generation (mpg). For example, the meaning procedures zones of need-v1 indicate that the complement of *need* has to be an infinitive clause (an xcomp). Xcomps can be of any complexity, which accommodates the generative nature of language: e.g., *He needs [to leave the studio before taping begins]*. With this background in place, let us proceed to explain the component tasks of rendering.

### 3.1 The Process of Scaffolding the GMR

The process of scaffolding the GMR [GMR-SCAFFOLDING] involves finding word senses in the lexicon to cover all event-based dependency structures in the GMR. Scaffolding captures the intuition that people make sense of complex entities by first understanding the big picture and then drilling down to the details. In GMRs, the big picture is the event dependencies and the details are their case-role fillers and modifiers. The search for event dependencies starts at the root of the GMR and works down through the layers of argument structures.

**Table 5.** Scaffolding the sentence *Mary needed to feed Fido before going out to dinner.*

| GMR | Lexical sense | Sem-struc zone |
|---|---|---|
| PRECEDES-1 | before-conj1 | PRECEDES |
| DOMAIN MODALITY-1 | | DOMAIN ^$var1 (EVENT/MODALITY) |
| RANGE EAT-AT-RESTAURANT-1 | | RANGE ^$var2 (EVENT/MODALITY) |
| MODALITY-1 | need-v1 | MODALITY |
| TYPE OBLIGATIVE | | TYPE OBLIGATIVE |
| VALUE 1 | | VALUE 1 |
| SCOPE FEED-1 | | SCOPE ^$var2 |
| TIME past | | ^$var2 |
| **FEED-1** | | AGENT ^$var1 |
| **AGENT HUMAN-1** | **feed-v1** | **FEED** |
| **BENEFICIARY DOG-1** | | **AGENT ^$var1** |
| TIME past | | **BENEFICIARY ^$var2** |
| HUMAN-1 | Mary-n1 | HUMAN |
| HAS-NAME Mary | | HAS-NAME Mary |
| DOG-1 | Fido-n1 | DOG |
| HAS-NAME Fido | | HAS-NAME Fido |
| EAT-AT-RESTAURANT-1 | go-v72 | EAT-AT-RESTAURANT |
| AGENT HUMAN-1 | | AGENT ^$var1 |
| TIME past | | |

The key to scaffolding is matching the largest GMR chunks possible at each level, as we will explain using the example in Table 1. To the left is the GMR for the example *Mary needed to feed Fido before going out to dinner*, and to the right are color-matched lexical senses whose sem-struc zones align with chunks of the GMR. The first layer of the GMR juxtaposes the two propositions: [Mary needed to feed Fido] before [going out to dinner]. That juxtaposition is established by the first sense of the conjunction *before* (before-conj1).[7] The second layer of the GMR covers the two frames in blue and can be accommodated by two senses: need-v1, which sets up the modality frame, and feed-v1, which specifies the event that is its scope. The final layer of this GMR is the headed by EAT-AT-RESTAURANT-1, which is matched by go-v72.

Scaffolding fulfills a practical necessity: it excludes lexical senses that incompletely cover functionally larger GMR chunks. For example, there is a nominal sense of *obligation* (obligation-n1) whose semantic interpretation is "MODALITY (TYPE OBLIGATIVE) (VALUE 1)". This does not leave room for the scope to be expressed — i.e., what event is obligatory. Since that *is* indicated in the GMR, the sense obligation-n1 will not be useful for building the sentence at hand.

Our top-down, largest-chunk search strategy operationalizes a theory of cognition we call *shapes of meaning*. It reflects the hypothesis that people remember and manipulate GMR chunks of various shapes and sizes, just as they remember and manipulate linguistic expressions of various shapes and sizes. Stated differently, neither reasoning nor language operates fully compositionally over the smallest possible atoms. A LEIA's ability to recognize and reuse complex structures, rather than always engage in first-principles reasoning, reduces cognitive and computational load, increases explanatory power, and captures human intuitions about how language and reasoning seem to work.[8]

Enabling the agent to recognize packages of knowledge leads to a trifurcation in the Scaffolding module. The dynamic scaffolding process we just described (DYNAMIC-SCAFFOLDING) is undertaken only if two kinds of psychologically plausible shortcuts are not available. The first involves utterances that have quasi-reflexive status: they are fixed, short, and frequent, such as "Duck!", "Ouch!", and "Need help?". Our previous example of asking "Which one?" is of this type. Identifying such fixed utterances is the first subevent of scaffolding: REFLEXIVE-SCAFFOLDING. If there is a match, then the language string is produced directly, which is a System 1 action similar to skill execution in robots (cf. Fig. 1). The second shortcut, SHAPE-MATCH-SCAFFOLDING, involves GMR shapes that are so standard and frequent that their scaffolding decisions have been pre-prepared. This is functionally similar to recording constructions in the lexicon not because they are semantically non-compositional but because they seem like standard structures that people access in full rather than recompute each time. If these scaffolding shortcut options do not yield a hit, then the agent undertakes the dynamic scaffolding process described above.

[7] There is a different sense of *before* that will generate the sentence *Mary needed to feed Fido before she went out to dinner*, and it, too, will be created during this process. During the last step in generation, a language model selects among the candidates.

[8] For related discussions see, e.g., Gentner and Smith (2013), Newell and Simon (1972), and Thalmann, Souza, and Oberauer (2019).

At the end of scaffolding, the agent has a list of senses (a sublexicon) for the heads of the dependency structures in the GMR. To fill out the scaffolding, it needs to find lexical senses that can be used to express the rest of the GMR constituents.

### 3.2 Filling Out the Scaffolding

Filling out the scaffolding is a script with the following five subevents.

**Transform Lexical Senses** (GEN-TRANSFORM). Earlier we explained what transformations are and why they are needed. It is at this point that the agent carries them out, replacing the original lexical senses with the transformed ones. In our feeding Fido example, the need for transformations was indicated in the meaning procedures zones of the lexical senses: need-v1 indicates that its complement has to be an infinitive (*needed to feed…*); and before-conj1, whose sense was not shown for reasons of space, indicates that the second conjunct's verb must be a present participle (before *going…*). However, there are other triggers for transformations as well, such as a mismatch between the syntactic requirements of an argument and the lexical senses available to realize it. For example, the verb *oppose* requires its object to be a noun phrase even if it conveys a proposition. So, one can say, *The boss opposed their leaving work early* or *The boss opposed the fact that they left work early,* but one cannot say **The boss opposed they left early.* The lexicon does not include nominalized forms of every verb, as that would be unnecessarily redundant. Instead, the GEN-TRANSFORM script scans for cases in which nominalization is needed and dynamically generates the needed nominalized senses.

**Freeze Lexicalized Head-Modifier Combinations** (FREEZE-HEAD-MOD-COMBOS). The sem-struc zones of lexical senses can include complex structures, such as concepts with modifiers. For example, *inspector* is described as HUMAN (AGENT-OF INSPECT), and *John* is described as HUMAN (HAS-NAME John). When a GMR chunk matches a complex sem-struc, that sense is selected and compositional alternatives are blocked — such as *a person who inspects* and *a person named John*.[9]

**Build Abstract Modified Scalars** (BUILD-MODIFIED-SCALARS). Scalar attributes can have values on the abstract scale {0,1} to express ideas like *tall, very tall, extremely tall, relatively short,* and so on. The lexicon includes words corresponding to some, but not all, such values: e.g., *tall* is expressed as "HEIGHT .8" and *short* is "HEIGHT .2", but other values need to be expressed using combinations of adverbs and adjectives: HEIGHT .9 "very tall", HEIGHT .3 "rather short". The program called in BUILD-SCALAR-MOD knits together adverbs like *very* and *rather* with appropriate adjectives.[10]

**Find Candidate Word Senses for As-Yet Untreated GMR constituents** (FINALIZE-GEN-SUB-LEX). By now, the agent has already identified lexical senses to treat many of the rows in the GMR. Now it searches the lexicon to account for what remains. This search is surfacy and is permitted to overshoot: the agent is not trying to optimize the syntactic and semantic fit of each candidate in the larger dependency structure, it is only collecting fodder for that process (i.e., Combinatorics).

---

[9] This algorithm will need to be enhanced since there are contexts in which one wants to say "a person named John" or "a person who inspects things". This example illustrates how the inventory of heuristics guiding system choices will continue to be expanded.

[10] If a scalar quantity between 0 and 1 is used as an actual value, then it should have an explicit or contextually recoverable measuring unit, such as .8 cm.

**Prepare Options for Referring Expressions** (PREPARE-REFEX-GEN). Selecting the best referring expression for a context (e.g., "the hammer", "it", "that", "the other one") is a complicated matter, and it would require a significant development effort to model it symbolically. The good news is that this is exactly the kind of task for which language models are well suited. However, to ensure that the language model will not hallucinate, we use knowledge-based reasoning to prepare the inventory of referring expressions that the language model can select from; it is not permitted to propose additional ones. Preparing this inventory involves things like freezing references to *I* and *you*, determining whether a noun phrase should have *a/an* or *the* based on its discourse status, reasoning about appropriate naming conventions for people based on their social relations to the agent (*Bob* vs. *the boss*), and preparing pronominal candidates depending on gender, number, animacy, politeness, object/event status, and so on.

By the end of the module *Filling Out the Scaffolding* the agent has a set of word senses that it will now combine into sentences.

### 3.3 Combinatorics

Creating combinations of lexical senses that result in legal sentences involves evaluating the syntactic and semantic constraints imposed by argument-taking words on their dependents. This process (GMR-COMBINATORICS) is conceptually similar for language generation and language understanding, and it is detailed for the former in McShane & Nirenburg (2021) and McShane, Nirenburg, & English (2024). Therefore, here we will mention only a few highlights.

Since the agent is using only the small sublexicon it has just compiled, combinatoric explosion is not expected to be an issue. However, for reasons of efficiency and ease of analyzing intermediate results, we include an initial step in which the agent assesses whether the sublexicon includes numerous strict synonyms — i.e., word senses for which the syntactic and semantic zones are identical, such as *sofa* and *couch*, or the transitive senses of *drink* and *sip.* If so, the agent temporarily prunes out all but one, carries out GMR-COMBINATORICS using just that option, and then reintroduces the strict synonyms back into the legal dependency structures.

The combinatoric process need not be guided by a cognitive model. Its output is what we call *near-sentences*, which include ordered lemmas appended with inflectional features. The remaining procedures — generating the correct inflectional forms, finalizing a few syntactic details (such as creating passive constructions), and selecting the best output sentence from among the candidates — could be modeled symbolically but need not be, since empirical methods are well suited to handling them.

### 3.4 Empirical Finalization

Various empirical tools can be used for the final steps of language generation (FINALIZE-GENERATION). For morphological generation and select syntactic processes (GEN-MORPH), we use SimpleNLG (Gatt & Reiter, 2009). For selecting the contextually best sentence from the set of candidates, a language model is appropriate. Although a *large* language model has performed well in our testing thus far (McShane, Nirenburg, and English, 2024), we hypothesize that a *small* language model would be sufficient and will be much preferable for reasons of speed, cost, and portability.

The reason why the LEIA typically has multiple candidates to begin with is the availability of paraphrases. For example: many objects and events have synonyms in the LEIA's lexicon; objects can be referred to using full referring expressions or pronouns; people can be referred to using their names or pronouns; utterances can be elliptical (*The red one?*) or fully specified (*Do you want the red hammer or the blue one?*); there are over a dozen ways to ask someone to do something (*Would you [do something]? Would you mind [doing something]? Could you [do something] for me? Please [do something], I'd appreciate it if you would [do something],* etc.); and so on.

Below are several examples that mock up how LEIA processing and LLM processing will interact when we have implemented the final selection processing during generation. The examples reflect an interaction with Claude.ai on 9/15/2026, formatted more concisely for presentation purposes. Each of the queries to Claude began with: "Below is a context in quotation marks. Select which of the numbered synonymous utterances that follow is best. Give only the number of the response, no explanation." The option Claude selected is underlined. The point is that the AI system selected a perfectly good synonym for each context, which is not surprising since this kind of surfacy manipulation of language is precisely what language models are good at.

**"I think I left my keys at home."**
1. Do you want us to look for your keys? 2. Do you want us to look for the keys? <u>3. Do you want us to look for them?</u> 4. Should we look for your keys? 5. Should we look for the keys? 6. Should we look for them?

**"'I think I left my keys at home.' 'Do you want us to look for them?' 'Of course. Why do you ask?' 'Because what you said was ambiguous: it could be a request or an assertion.' 'OK. Yes, look for them.'"**
1. What do your keys look like? 2. What do the keys look like? <u>3. What do they look like?</u> 4. Describe your keys. 5. Describe the keys. 6. Describe them.

**"'I think I left my keys at home.' 'Do you want us to look for them?' 'Of course. Why do you ask?' 'Because what you said was ambiguous: it could be a request or an assertion.' 'OK. Yes, look for them.' 'What do they look like?' 'They are on a red keychain with a small flashlight.'"**
1. Do you recall where you last had them? 2. Do you recall where you last had the keys? 3. Do you recall where you last had your keys? <u>4. Where did you last see them?</u> 5. Where did you last see the keys? 6. Where did you last see your keys?

**"'Lube oil pressure is at 3.8 bar.' 'Why are you telling me that?'"**
1. It's a little low. <u>2. It's slightly low.</u> 3. The reading is a little low. 4. The reading is slightly low. 5. The reading is lower than it should be. 6. It's lower than it should be. 7. The reading is lower than normal. 8. It's lower than normal.

There is little risk that the empirical tools that we use during Empirical Finalization will cause damage because they are not being permitted to modify *meaning*, they are only manipulating the surface forms of words and selecting from fully-formed sentence options. Even if the final sentence the language model selects is not what a person would have preferred, it will still be correct because

it was among the semantically correct sentences the LEIA generated. Furthermore, if the choice unwittingly introduces ambiguity, as by including a pronoun that would better have been expressed using a full noun phrase, then the human interlocutor can ask for clarification in the normal flow of communication. The fact that LEIAs are being designed to engage in natural communication cannot be overemphasized. For social robots to serve as useful, natural assistants to people, they must be excellent at a broad range of social skills centrally involving collaboration, learning, inferencing, and mindreading.

## 4 Implementation

In terms of the *theory-model-system* trichotomy that is central to LEIA modeling (McShane, Nirenburg, & English, 2024, section 2.3, which builds upon Bailer-Jones, 2009), what we have described is a *model* of language generation grounded in the *theory* of Ontological Semantics. The model can be implemented in any cognitive *system* compatible with the foundational principles of LEIAs.

In order for cognitive modeling to be a proper contribution to AI, it must be implementable, which can be demonstrated in proof-of-concept systems. We have completed the initial implementation of this model, which will be incorporated into all LEIA cognitive robots moving forward. This implementation uses the existing lexicon and ontology, and already covers several dozen generation and rendering phenomena such as infinitival complements, relative clauses, participial complements, and copular sentences. Like all microtheories, it will be enhanced over time.[11]

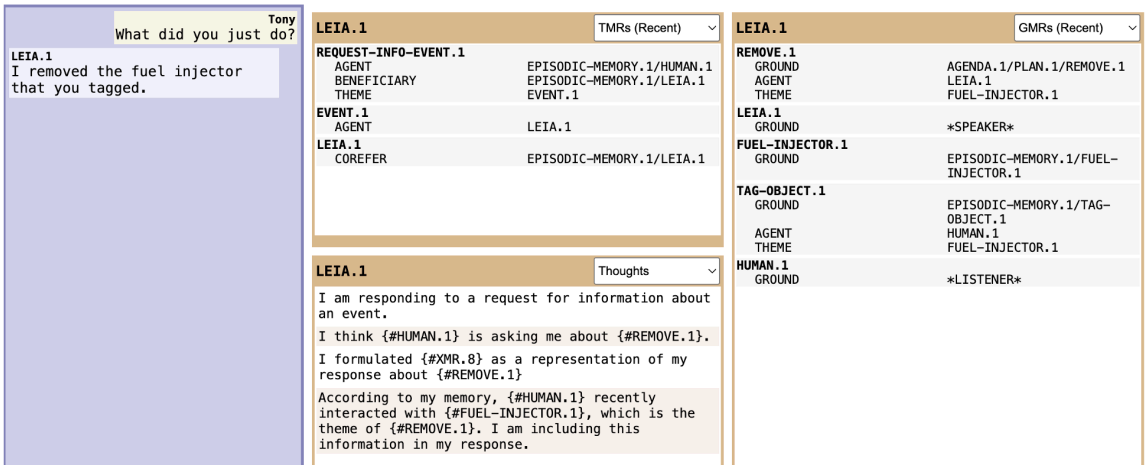



**Fig. 2.** Tracing language generation in the under-the-hood panels of the simulation environment.

Fig. 2 shows under-the-hood panels (i.e., dynamic, human-readable traces of system operation) from the LEIA simulation environment. In this example, right after the LEIA robot removes the fuel injector, the person, Tony, asks, `What did you just do?` The TMRs panel shows the

[11] Some algorithms, as for generating the inventory of referring expression variants, are currently implemented in a simplified manner.

LEIA's interpretation of this sentence. The Thoughts panel traces its thinking about how to respond. The GMRs panel shows the meaning representation that is subsequently rendered into English as `I removed the fuel injector that you tagged`. This example expands upon an existing simulation system in which a LEIA assists a person in replacing a fuel injector on a ship (McShane et al., 2026a). Fig. 3 shows additional details about the generation process in the DEKADE developers' environment, which enables knowledge acquisition, testing, and debugging. The details don't matter; what does is the fact that modern computational tools have created a completely new world for building symbolic systems, in contrast to symbolic modeling half a century ago.

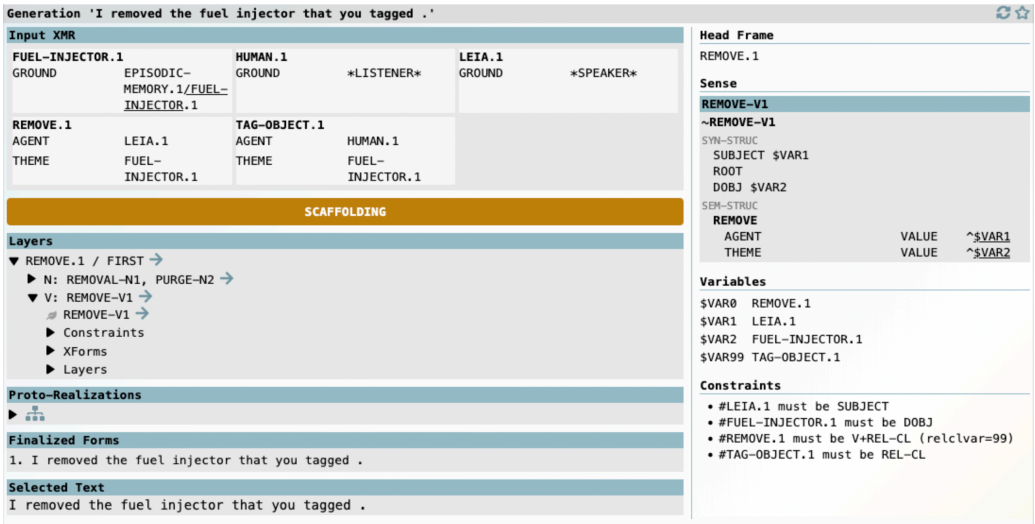



**Fig. 3** Observing traces of the language generation process in the DEKADE development environment.

At the conference we will demonstrate language generation in a simulation system (Fig. 4) whose operation is detailed in McShane et al. (2026a). In that scenario, two LEIA robots, an unmanned ground vehicle and a drone, search for missing keys in an apartment. The right two frames are among the many under-the-hood panels available in the environment, which show human-readable traces of agent functioning for purposes of validation and demonstration.

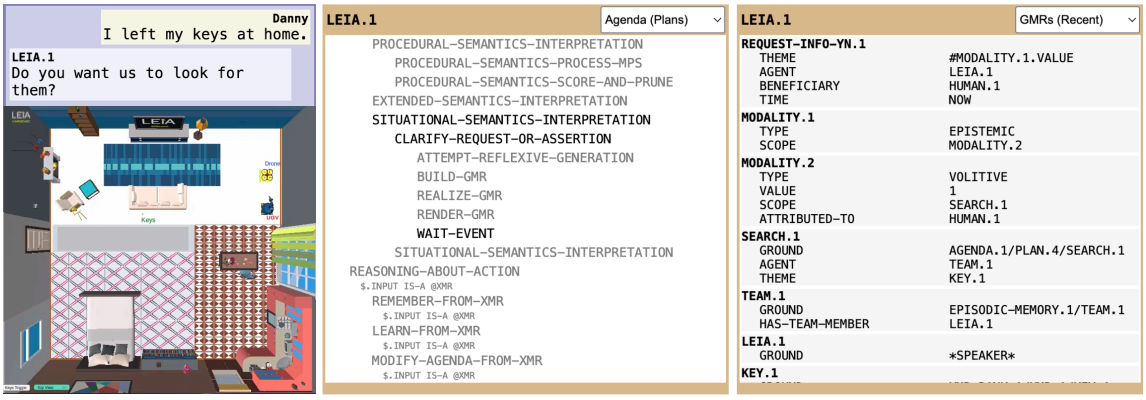



**Fig. 4.** A view of the key search simulation that will be shown at the conference.

## 5 Comparisons with Others

We do not know of any work on generation that is similar enough in kind or objective to support a genuinely interesting comparison with what we have presented here, but more distant resonances provide some context. Early work on language generation, which dwindled around the turn of the century, is well represented by Reiter and Dale (2000), who analyze the challenges of language generation and discuss approaches to overcoming them that were implemented in systems of that era. Their framework accommodates systems using any kind of input, from unstructured data through formal meaning representations. By contrast, LEIAs always generate language from meaning representations written in the formal language of the ontology. Reiter and Dale do not treat generation from meaning representations with nearly the degree of detail we provide here.

Although most cognitive systems require language processing, most cognitive systems research focuses on matters other than language per se and, understandably, relies on various stopgaps for communication in proof-of-concept systems. For example, in their contribution on abductive inference using domain-level and meta-level knowledge, Gabaldon, Langley, and Meadows (2013) explain, "… [O]ur system does not attempt to model speech or sentence processing. Rather, it assumes the logical meanings of utterances as inputs and outputs, supporting both dialogue interpretation and generation at this abstract level." Another example is Mohan et al. (2012), which describes an interactive learning system whose generation system is described simply as "Messages from the agent are converted to natural language using templates." Our research group also deemphasized generation until language understanding and various other core agent capabilities were sufficiently developed. Our reason for citing these examples is to point out that when symbolic natural language generation appears in cognitive systems reports, it is almost never itself a research contribution.

Beyond systems-level work, an occasional paper addresses an individual, theoretically interesting aspect of symbolic generation, but not within a robust, end-to-end generation system (e.g., Briggs & Scheutz, 2016; Briggs et al., 2022). Insights from such work might be useful for LEIA-style generation if they were packaged as microtheories with a clearly defined role in the agent's generation pipeline and with implemented methods for satisfying prerequisites.

A reviewer of this paper wrote, “I feel the literature on AMR-to-text generation should be discussed, as it’s closely related to the proposed approach.” We actually don’t think it is, but this offers us a good opportunity to explain why.

AMR — Abstract Meaning Representation — is a semantically shallow corpus annotation schema aimed at supporting treebanking for machine learning (Banarescu et al., 2013). A handful of features of AMR should suffice to contrast it with the meaning representations of Ontological Semantics. (1) The AMR scheme had to be simple enough to support fast and reliable annotation; it is not based in a comprehensive theory of semantics and pragmatics. (2) AMR annotation is done on individual sentences, with no incorporation of context. (3) “AMR is heavily biased towards English. It is not an Interlingua… AMR concepts are either English words (“boy”), PropBank framesets (“want-01”), or special keywords” (Banarescu et al., 2013). (4) AMRs use numbers for case roles rather than semantically meaningful relations, such as agent and theme. (5) AMRs do not include inflectional morphology and, therefore, lose the associated meanings. (6) AMRs omit articles, which means that they lose associated aspects of coreference (the many uses of the article *the* in English are described in McShane and Nirenburg, 2021). (7) AMRs do not sufficiently treat modality. (8) AMRs fall back on English lemmas when an input string is not covered by PropBank. (9) “AMR… remain[s] agnostic about the relation between strings and their meanings, considering this a topic of open research” (Banarescu et al., 2013).

To overcome some of the limitations of AMR — without veering from the focus of manual corpus annotation — the Uniform Meaning Representations (UMR) language was developed (Bonn et al., 2024). Improvements over AMR included adding aspect, person, number, and quantification; going beyond sentence boundaries with select relations; and supporting the annotation of languages apart from English.

AMRs were, in fact, used as input to machine-learning-based text generation (Mager et al. 2020); however, the contribution of that work was squarely in the realm of machine learning techniques, not linguistic content or the computational modeling of cognition.

The program of research underlying AMR/UMR made two assumptions: that AMR/UMR graphs reflected enough of the meaning of texts to be useful in language processing applications, and that having enough such representations, acquired by hand, would lead to noticeably better language processing applications, as judged by benchmarks. Both assumptions have been undermined by the current availability of LLMs. By contrast, Ontological Semantics has not been superseded in this way by LLMs because it never shared AMR/UMR’s goals or methods. Where AMR and UMR were designed to produce corpus annotation at scale for ML training, Ontological Semantics does not involve corpus annotation at all. Although Ontological Semantics requires knowledge bases, their acquisition is done in part manually, in part semi-automatically, and in part through lifelong learning by agents themselves. The GMRs that drive generation are produced on the fly by LEIAs as part of their cognition. The AMR/UMR program and the LLM revolution both belong to the same engineering tradition — the pursuit of benchmark performance through data-driven statistical learning. Ontological Semantics belongs to a different tradition entirely: the pursuit of meaning-oriented computational cognition.

## 6 Final Thoughts

A reviewer wrote, "the work is not aimed at improving our understanding of human cognition." In fact, it is. LEIA models of language understanding and generation provide more insight into comprehensive, end-to-end human language processing than anything else proposed to date. Even the theoretical and cognitive linguistic research, whose objective is to provide insights into human cognition, falls short when it comes to language — as detailed in McShane, Nirenburg, and English (2024, section 4.4). For example, generative grammar is not computationally tractable, it treats only syntax (not semantics or pragmatics), and it does not include end-to-end algorithms — it only tests the grammaticality of fully-formed sentences. Construction grammar, for its part, only lightly incorporates semantics, remaining largely a syntactic theory. Moreover, what is called semantic analysis is typically only an English paraphrase of the given expression. In most flavors of construction grammar, there are no algorithms for text understanding or generation. In the thread called *embodied construction grammar*, most of the work to date has been on computer science angles of embodied cognition, not algorithms capturing symbolic modeling that could guide computational language understanding or generation.

Language processing by LEIAs, by contrast, treats a very large number of linguistic and extralinguistic phenomena and their interactions. LEIA modeling does not attempt to exactly mimic what people do because nobody knows what people do, and attempting to reproduce human wetware is a non-starter. For example, one might claim that people have expectations about what other people will say before they say it based on theories of priming. However, these expectations are often mistaken — as can be seen when someone tries to finish someone else's sentence but gets it wrong. It makes more sense, at least at the current state of the art, to have agents process what is actually said. This doesn't mean that LEIAs don't have expectations: they do, recorded in the scripts they are following; and these expectations help with resolving things like ambiguity and ellipsis. Finally, the fact that many human experiments on priming have not been reproducible has significantly undermined that theory. This is one of many reasons why it is important to rigorously and continuously analyze what counts as scientifically-derived knowledge. Clearly, human experiments are not the sole metric of progress in understanding human cognition.

When one uses demonstration systems as proof of concept, there must be a clear path to scaling up. For LEIA development, we have one. We have published extensively on cognitive modeling methodologies, including knowledge acquisition that incorporates manual, semi-automatic, and automatic methods. In fact, lifelong learning by LEIAs is a current emphasis of our R&D (McShane, Nirenburg, and English, 2024; Arndt, Nirenburg, and Oruganti, 2026; Nirenburg et al., 2026). The main barrier to scaling up fast is the need for sufficient bootstrapping knowledge to support the agent's independent learning. For example, whereas agents can learn new objects, physical events, and scripts involving physical events, it is considerably harder to prepare them to learn abstract concepts and elements of the closed-class lexicon (for discussion, see McShane, Nirenburg, and English, 2024, Chapter 9).

The need to record specific knowledge to support specific agent cognition might appear inelegant to researchers accustomed to correlating good science with the most all-encompassing theory or the most generic problem-solving algorithm. However, such preferences carry a cost, as is well

demonstrated by language models, which use a wholly generic problem-solving algorithm whose operation is not inspectable and whose operation is error-prone.

The expense of knowledge engineering needed to build LEIA-style systems is quite comparable to that of engineering generative AI systems. Concerns about the human labor required for symbolic modeling are most fairly assessed with reference to (a) actual empirical-AI expenditures on human labor (data cleaning, tagging, prompt engineering, etc.), (b) the promise of independent, lifelong learning by LEIAs, and (c) the promise of having truly reliable, explainable AI systems.

**Acknowledgements**

This research was supported in part by Grants N00014-16-1-2118 ($1,317,117) and N00014-17-1-2218 ($1,499,352) from the U.S. Office of Naval Research. Any opinions or findings expressed in this material are those of the authors and do not necessarily reflect the views of the Office of Naval Research.

**References**

Arndt, C., Nirenburg, S., & Oruganti, S. (2026). A script for learning scripts: Automatic acquisition of procedural knowledge by ontologically grounded agents. *Proceedings of the 48th Annual Meeting of the Cognitive Science Society*, Rio de Janeiro, Brazil. July.

Bailer-Jones, D. M. (2009). *Scientific Models in Philosophy of Science*. University of Pittsburgh Press.

Banarescu, L., Bonial, C., et al. (2013). Abstract meaning representation for sembanking. *Proceedings of the 7th Linguistic Annotation Workshop & Interoperability with Discourse*, pp. 178–186. Association for Computational Linguistics.

Bonn, J. Buchholz, M.J., Chun, J., et al. (2024). Building a broad infrastructure for Uniform Meaning Representations. *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 2537–2547.

Briggs, G., & Scheutz, M. (2016). The pragmatic social robot: Toward socially-sensitive utterance generation in human-robot interactions. *Proceedings of the 2016 AAAI Fall Symposium Series: Artificial Intelligence for Human-Robot Interaction*, Technical Report FS-16-01.

Briggs, G., Williams, T., Jackson, R. B., & Scheutz, M. (2022). Why and how robots should say 'no'. *International Journal of Social Robotics, 14*(2), 323–339. https://doi.org/10.1007/s12369-021-00780-y

Gabaldon, A., Langley, P., & Meadows, B. (2013). Integrating meta-level and domain-level knowledge for interpretation and generation of task-oriented dialogue. *Proceedings of the Second Annual Conference on Advances in Cognitive Systems,* pp. 171–186.

Gatt, A., & Reiter, E. (2009). SimpleNLG: A realisation engine for practical applications. *Proceedings of the 12th European Workshop on Natural Language Generation*, *March*, pp. 90–93.

Gentner, D. & Smith, L. A. (2013). Analogical learning and reasoning. In D. Reisberg (Ed.), *The Oxford Handbook of Cognitive Psychology*, 668–681. New York, NY: Oxford University

Press.

Mager, M., Astudillo, R.F., Naseem, T., et al. (2020). GPT-too: A language-model-first approach for AMR-to-text generation. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1846–1852. Association for Computational Linguistics.

McShane, M., English, J., Arndt, C., Nirenburg, S., & Oruganti, S. (2026a). Explaining cognition and communication in cognitive robots. *Proceedings of METACOG-26: Symposium on Artificial Metacognition*. AAAI Fall Symposium Series.

McShane, M., & Leon, I. (2021). Language generation for broad-coverage, explainable cognitive systems. *Proceedings of the Ninth Annual Conference on Advances in Cognitive Systems.*

McShane, M., & Nirenburg, S. (2021). *Linguistics for the Age of AI*. MIT Press.

McShane, M., Nirenburg, S., & Beale, S. (2005). Semantics-based resolution of fragments and underspecified structures. *Traitement Automatique des Langues,* 46(1): 163–184.

McShane, M., Nirenburg, S., & English, J. (2024). *Agents in the long game of AI: Computational cognitive modeling for trustworthy, hybrid AI*. MIT Press.

McShane, M., Nirenburg, S., English, J., & Oruganti, S. (2025a). Pursuing actionable perception interpretation in cognitive robotic systems. *Proceedings of the Twelfth Annual Conference on Advances in Cognitive Systems*, pp. 367–386.

McShane, M., Nirenburg, S., English, J., & Oruganti, S. (2026b). A cognitive model and cognitive rationale for natively multilingual robots. *Proceedings of the Thirteenth Annual Conference on Advances in Cognitive Systems*.

McShane, M., Nirenburg, S., Oruganti, S., & English, J. (2025b). An ecosystem for scalable symbolic modeling in neurosymbolic AI; or Shapes of cognition. *Proceedings of the Twelfth Annual Conference on Advances in Cognitive Systems*, pp. 45–64.

Mohan, S., Mininger, A. H., Kirk, J. R., & Laird, J. E., (2012). Acquiring grounded representations of words with situated interactive instruction. *Proceedings of Advances in Cognitive Systems*.

Newell, A., & Simon, H. A. (1972). *Human Problem Solving.* Prentice-Hall.

Nirenburg, S., McShane, M., Oruganti, S., English, J., & Arndt, C. (2026). LLMs as knowledge workers. *Proceedings of MAKE-2026.* AAAI Spring Symposium. Burlingame, CA.

Nirenburg, S., & Raskin, V. (2004). *Ontological Semantics*. MIT Press.

Oruganti, S., Nirenburg, S., McShane, M., English, J., Roberts, M., Arndt, C., Parasuraman, R., & Sentis, L. (2026). Why cognitive robotics matters: Lessons from OntoAgent and LLM deployment in HARMONIC for safety-critical robot teaming. *arXiv:2603.26730*.

Reiter, E., & Dale, R. (2000). *Building Natural Language Generation Systems*. Cambridge University Press.

Thalmann, M., Souza, A.S., & Oberauer, K. (2019). How does chunking help working memory? *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 45(1): 37–55.